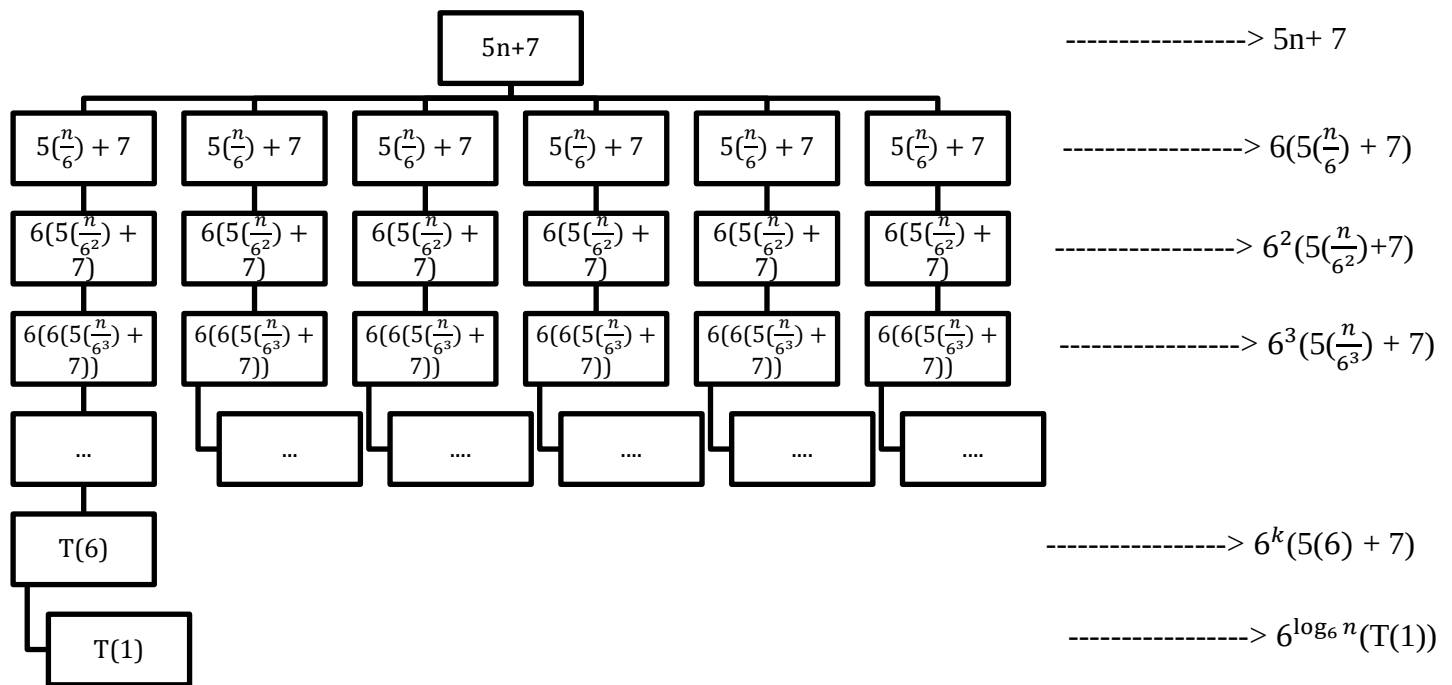


Assignment 3

- (5 pts) Suppose that $T(n) = 3$ for $n = 1$, and for all $n \geq 2$, a power of 6, $T(n) = 6T(n/6) + 5n + 7$. Solve this recurrence exactly by drawing a **recursion tree**. (You will need to give an exact explicit solution for T , by specifying all coefficient and constant values, do not use O , Ω , or Θ). You can assume that $n = 6^i$ for some non-negative integer i .

$$T(n) = \begin{cases} 3, & n = 1 \\ 6T\left(\frac{n}{6}\right) + 5n + 7, & x \geq 2 \end{cases}$$



Solve for k : $\frac{n}{6^k} = 6$

$$\log_6 \frac{n}{6^k} = \log_6 6 \rightarrow \log_6 n - \log_6 6^k = 1 \rightarrow \log_6 n - k = 1 \rightarrow k = \log_6 n - 1$$

$$T(n) = \sum_{i=0}^{\log_6 n - 1} 6^i \left(5\left(\frac{n}{6^i}\right) + 7 \right) + 6^{\log_6 n} (T(1)) = \sum_{i=0}^{\log_6 n - 1} 5n + 6^i(7) + 6^{\log_6 n} (T(1))$$

$$T(n) = 5n \sum_{i=0}^{\log_6 n - 1} 1 + 7 \sum_{i=0}^{\log_6 n - 1} 6^i + 6^{\log_6 n} (T(1))$$

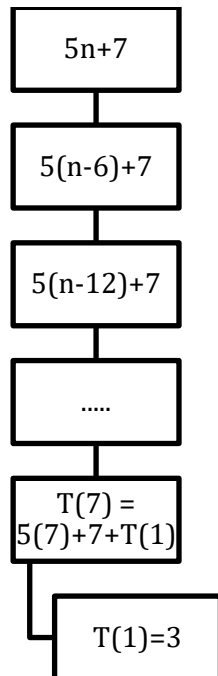
$$T(n) = 5n(\log_6 n - 1 + 1) + 7 \left(\frac{6^{\log_6 n - 1 + 1} - 1}{6 - 1} \right) + n(T(1))$$

$$T(n) = 5n(\log_6 n) + 7\left(\frac{n-1}{5}\right) + n(T(1)) = 5n(\log_6 n) + \frac{7n-7}{5} + 3n \rightarrow \text{fastest term} = 5n(\log_6 n)$$

Assignment 3

2. (5 pts) Suppose that $T(n) = 3$ for $n = 1$, and for all $n \geq 2$, $T(n) = T(n-6) + 5n + 7$. Solve this recurrence exactly by drawing a **recursion tree**. (You will need to give an exact explicit solution for T , by specifying all coefficient and constant values, do not use O , Ω , or Θ). You can assume that $n = 6*i + 1$ for some non-negative integer i .

$$T(n) = \begin{cases} 3, & x = 1 \\ T(n-6) + 5n + 7, & x \geq 2 \end{cases}$$



Find upper bound of summation:

$$5(6i+1)+7 = 5n+7$$

$$5+30i+7 = 5n+7$$

$$5+30i = 5n$$

$$30i = 5n-5$$

$$i = \frac{n-1}{6}$$

$$T(n) = T(1) + \sum_{i=1}^{\frac{n-1}{6}} 5(6i+1) + 7 = T(1) + \sum_{i=1}^{\frac{n-1}{6}} 30i + 5 + 7 = T(1) + \sum_{i=1}^{\frac{n-1}{6}} 30i + 12$$

$$T(n) = T(1) + 30 \sum_{i=1}^{\frac{n-1}{6}} i + 12 \sum_{i=1}^{\frac{n-1}{6}} 1 = T(1) + 30 \left(\frac{\frac{n-1}{6}(\frac{n-1}{6}+1)}{2} \right) + 12 \left(\frac{n-1}{6} \right)$$

$$T(n) = T(1) + \frac{5n^2}{12} + \frac{11n}{3} - \frac{49}{12}$$

$$T(n) = \frac{5n^2}{12} + \frac{11n}{3} - \frac{13}{12}$$

$$\text{Fastest term} = \frac{5n^2}{12} \rightarrow n^2$$

Assignment 3

3. (3 pts) Let $T(n) = 3$ for $n = 1$, and for all $n \geq 2$, $T(n) = T(n-6) + 5n + 7$. Solve this recurrence using the mathematical induction method. (i.e., You need to guess a solution – explicit solution of $T(n)$ (you need to specify all coefficients, do not use O , Ω , or Θ), and assume that the guess is true for $n=k$. Then prove that it is also true for $n=k+6$. You also need to prove that your guess is true for the base case.)

$$\text{Claim } T(k) = \frac{5k^2}{12} + \frac{11k}{3} - \frac{13}{12}$$

Base Case: when $n = 1$

$$T(1) = \frac{5(1)^2}{12} + \frac{11(1)}{3} - \frac{13}{12} = 3$$

Induction step: Assume $T(k) = \frac{5k^2}{12} + \frac{11k}{3} - \frac{13}{12}$ is true for $n = k$. Then we need to prove that our guess is true for $n = k+6$.

$$T(k+6) = \frac{5(k+6)^2}{12} + \frac{11(k+6)}{3} - \frac{13}{12} = \frac{5k^2+60k+180}{12} + \frac{44k+264}{12} - \frac{13}{12} = \frac{5k^2}{12} + \frac{26k}{3} + \frac{431}{12}$$

$$T(k+6) = T(k+6-6) + 5(k+6) + 7$$

$$T(k+6) = T(k) + 5k + 30 + 7$$

$$T(k+6) = \frac{5k^2}{12} + \frac{11k}{3} - \frac{13}{12} + 5k + 30 + 7$$

$$T(k+6) = \frac{5k^2}{12} + \frac{26k}{3} + \frac{431}{12}$$

Thus our guess is true for $n=k+6$ if it is true for $n = k$.

4. (3 pts each – 9 pts total) Use the master method to give tight asymptotic bounds (Θ bound) for the following recurrences. (Specify, a , b , $f(n)$, c , and ε value when they apply.)

1. $T(n) = 8T(n/2) + 7n$

$a = 8, b = 2, f(n) = 7n$

$$\log_b a = \log_2 8 = \log_2 2^3 = 3 \log_2 2 = 3$$

$f(n) = 7n, n = n^3 \rightarrow \text{case 1}$

$$f(n) = O(n^{\log_b a - \varepsilon}) = O(n^{\log_2 8 - \varepsilon}) = O(n^{3 - \varepsilon}), \varepsilon = 1 \quad O(n^2)$$

$$T(n) = \Theta(n^3)$$

Assignment 3

$$2. \quad T(n) = 3T(n/9) + 4n^2$$

$$a = 3, b = 9, f(n) = 4n^2$$

$$\log_b a = \log_9 3 < \log_9 9 \rightarrow \text{less than 1}$$

$$f(n) = 4n^2 > n = n^1 > n^{\log_9 3} \rightarrow \text{case 3}$$

$$f(n) = \Omega(n^{\log_9 3 + \epsilon}), \epsilon = 1 - \log_9 3 > 0$$

$$af\left(\frac{n}{b}\right) \leq cf(n) \text{ for } 0 < c < 1 \text{ and } n \geq b$$

$$3f\left(\frac{n}{9}\right) = 3\left(4\left(\frac{n}{9}\right)^2\right) = 12\left(\frac{n^2}{81}\right) = \frac{4}{27}n^2$$

$$\frac{4}{27}n^2 \leq c(4n^2) \text{ if } c = \frac{1}{27} < 1$$

$$T(n) = \Theta(4n^2)$$

$$3. \quad T(n) = 9T(n/3) + 5n^2$$

$$a = 9, b = 3, f(n) = 5n^2$$

$$\log_b a = \log_3 9 = \log_3 3^2 = 2\log_3 3 = 2$$

$$f(n) = 5n^2, n = n^2 \rightarrow \text{case 2}$$

$$\Theta(n^{\log_b a} \log(n)) = \Theta(n^2 \log(n))$$

5. (8pts) Please see the attached [ompProb.cpp](#) file, and complete the program by writing code to compute the count of larger integers (larger than 20) among randomly generated numbers by:

a) complete largerCountWithLoop function to compute the count of larger integers (larger than 20) in the array using a for loop sequentially. Include your code for this function in the file that you will be submitting.

```
int largerCountWithLoop(int * A, int n)
{
    //counter for number of numbers in array larger than 20
```

Assignment 3

```
int count = 0;

//loop through entire array, size n --> number of variables to be randomly generated
for(int i = 0; i < n; i++)
{
    //check if the current position in array's number is greater than 20
    if(A[i] > 20)
    {
        //increase count if greater than 20
        count++;
    }
}
return count;
}
```

b) complete largerCountWithLoop_OMP function to compute the count of larger integers (larger than 20) in the array using a for loop in parallel (using OMP). Include your code for this function in the file that you will be submitting.

```
int largerCountWithLoop_OMP(int * A, int n)
{
    //counter for number of numbers in array larger than 20
    //defining i so that it can be used in parallel
    int i, count = 0;

    //indicating multithreading
    //indicates the shared values and private values between the threads
    #pragma omp parallel shared(A, n) private(i)

    //for loop will be divided among threads
    //at end of parallel, count will be reduced using + operation
    #pragma omp for reduction(+:count)
    //loop through entire array, size n --> number of variables to be randomly generated
    for(i = 0; i < n; i++)
    {
        //check if the current position in array's number is greater than 20
        if(A[i] > 20)
        {
            //increase count if greater than 20
            count++;
        }
    }
    return count;
}
```

Rachel Guzmán
MW 10:30-11:45
rguzma15@asu.edu

c) Using 4 threads, execute your program using 100, 10000, 1000000, and 10^8 as the values of n (the number of randomly generated numbers and also the array size). Then record the execution time for each of the four functions above in a table format:

execution time	n=100	n=10000	n=1000000	n=100000000
largerCountWithLoop	0.000002	0.000065	0.006151	0.628361
largerCountWithLoop_OMP	0.000211	0.004889	0.005920	0.232594

After completing your function, you can complete it in general.asu.edu as:

```
g++ -fopenmp -o ompProb ompProb.cpp
```

Then set the number of threads to 4 as:

```
export OMP_NUM_THREADS=4
```

Then execute it as:

```
./ompProb
```